Electrical and Computer Engineering Department

First Semester 2025/2026

ENCS4320: Applied Cryptography

HW#2

Deadline: 27/01/2026

Design and Implementation of a Secure Communication Protocol

1. Project Overview

This hands-on project guides you through building a secure communication system from the ground up. You'll start with an insecure plaintext protocol and progressively enhance it with modern cryptographic techniques. By completion, you'll have implemented a production-grade encrypted communication framework demonstrating real-world security engineering principles.

Personalization Requirement: All cryptographic materials will be uniquely generated using your Student ID, ensuring each implementation is individualized while maintaining verifiable correctness.

---

2. Phase I: Establishing Encrypted Communication

Goal: Implement confidential message exchange using symmetric encryption to protect against eavesdropping.

2.1 Cryptographic Components

Encryption Algorithm: AES (Advanced Encryption Standard)

- **Key Size**: 128 bits (16 bytes)
- **Mode**: CBC (Cipher Block Chaining) - ensures identical messages produce different ciphertexts
- **Padding**: PKCS#7 - automatically adjusts message length to required block size (16 bytes)
- **IV (Initialization Vector)**: 16 random bytes generated fresh for each message

2.2 Generating Your Personal Cryptographic Materials

Your unique Student ID determines your cryptographic parameters:

Step 1 - Derive Your AES Key:

```
AES_Key = MD5(Your_Student_ID)
Result: 16-byte key unique to you
```

Step 2 - Derive Your Reference IV:

```
Reference_IV = First 16 bytes of SHA256(Your_Student_ID)
```

Important: While the reference IV is deterministic for testing, all subsequent messages must use randomly generated IVs to maintain security.

2.3 Implementation Checklist

- Generate your personal AES key from your Student ID
- Implement AES-CBC encryption with PKCS#7 padding
- Generate fresh random IV for each encrypted message
- Transmit IV alongside ciphertext (IV is not secret)
- Successfully decrypt messages at the receiving end
- Verify encrypted traffic shows no plaintext in network capture

---

3. Phase II: Key Exchange and Authentication

Goal: Solve the key distribution problem and verify identities using public-key cryptography.

3.1 Authentication Infrastructure

Digital Signatures (RSA-128):

- Each participant generates an RSA key pair (128-bit keys)
- Public keys are linked to Student IDs to establish identity
- Private keys sign messages to prove authenticity
- Recipients verify signatures before trusting received data

Security Benefit: Prevents impersonation attacks - only the legitimate Student ID holder can produce valid signatures.

3.2 Session Key Agreement Protocol

Objective: Generate a shared secret key without transmitting it directly.

Protocol Flow:

1. **Setup Phase**
    - Both parties agree on Diffie-Hellman parameters (prime `p` and generator `g`)
    - Each party generates a random private value
2. **Exchange Phase**
    - Each party computes their public DH value
    - Public values are signed with RSA private keys
    - Signed public values are exchanged

3. **Verification Phase**
    - Each party verifies the received signature using the sender's RSA public key
    - Verification confirms the sender's identity
4. **Key Derivation**
    - Both parties independently compute the shared secret `s`
    - Session key derived via: `Session_Key = SHA256(s)[0:16]`
    - This session key replaces the static Phase I key

Critical Security Property: Even if your long-term RSA keys are later compromised, past session keys remain secure (Perfect Forward Secrecy).

---

4. Security Guarantees

Your implementation must achieve:

Property	Description	Validation Method
Confidentiality	Attackers monitoring network traffic cannot read messages	Network packet analysis shows only ciphertext
Integrity	Any tampering with messages is detected	Modified ciphertexts fail to decrypt properly
Authenticity	You can verify who sent each message	Signature verification confirms sender identity
Forward Secrecy	Past sessions remain secure even if keys leak later	Each session uses ephemeral keys

---

5. Deliverables

5.1 Source Code

- Clean, well-commented implementation
- Separate modules for: cryptography, networking, protocol logic

5.2 Technical Validation Report

"Students are required to submit a report detailing their workflow, supported by screenshots for every step."

5.3 Network Capture

- Submit a `.pcap` file showing:

- Complete Phase II handshake (key exchange with signatures)
- Encrypted message exchange using session keys
- Annotations identifying protocol phases